DEPI · SYSTEM ANALYSIS & DESIGN · DELIVERABLE 5 OF 6

System Deployment & Integration

Section 4.5 — Technology stack, deployment diagram, component diagram

---

MediCare — Clinic Management & Appointment System

ASP.NET Core MVC · N-Tier · Microsoft Azure

Team: Mohamed Munir (Team Lead) · Malak Hany · Ashraf Ibrahim · Leqaa Ibrahim · Natalie Bola

November 2026

MediCare – Technology Stack

Technology Stack

1. Technologies by Layer

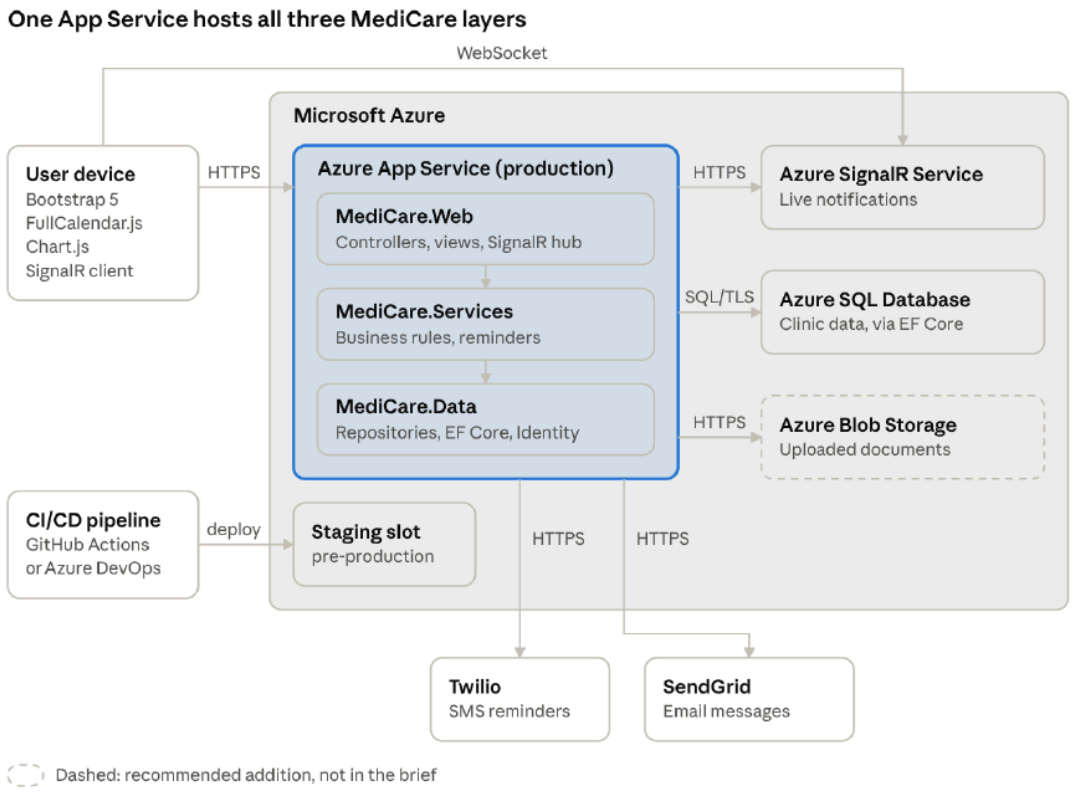
Layer / area	Technology	Why it was chosen
Presentation: framework	ASP.NET Core MVC	Native MVC support, high performance, and excellent integration with Identity, SignalR and built-in DI.
Presentation: UI	Razor Views + Bootstrap 5	Responsive, fast to build, ready-made components, and dark/light mode support.
Presentation: calendar	FullCalendar.js	Interactive calendar for showing available slots and managing appointments (day / week / list views), with events loaded via AJAX.
Presentation: charts	Chart.js	Lightweight, simple charts for appointment statistics, revenue and popular specializations.
Presentation: real-time	ASP.NET Core SignalR (+ Azure SignalR Service)	Instant notifications to doctors and patients without refreshing, and scalable in production.
Business logic	C# / .NET (services + DTOs)	Strongly typed language, with services split by domain.
Business logic: mapping and validation	AutoMapper + FluentValidation	Less manual mapping code and clear, centralized validation rules.
Business logic: background jobs	Hosted Services / Hangfire	Reliable scheduling of the 24-hour appointment reminders.
Data access: ORM	Entity Framework Core (Code-First + Migrations)	Defines the schema in code, manages database changes, and LINQ queries are safe from SQL injection.
Data access: patterns	Generic Repository + Unit of Work	Separates data access from business logic and guarantees consistent transactions.
Database	Azure SQL Database (SQL Server locally during development)	A relational database suits the connected data (patients, doctors, appointments), with backups and managed scaling.
Authentication and authorization	ASP.NET Core Identity	Manages users and the three roles, hashes passwords, and integrates directly with MVC.
SMS notifications	Twilio	Reliable API for sending SMS reminders internationally.
Email notifications	SendGrid	Templated email delivery with delivery tracking.
Reports / export	QuestPDF (or iTextSharp) + ClosedXML	Exports reports as PDF and Excel.
File storage	Azure Blob Storage	Secure, scalable storage for test results, X-rays and medical documents.
Logging and monitoring	Serilog + Azure Application Insights	Tracks errors and performance in production.
Secrets	Azure Key Vault	Protects Twilio and SendGrid keys and connection strings.
Testing	xUnit + Moq (+ EF Core InMemory)	Tests services and conflict-detection logic without a real database.
Hosting	Microsoft Azure App Service	Managed deployment, auto-scaling, and integration with SQL and SignalR Service.
CI/CD	GitHub Actions or Azure DevOps Pipelines	Automatic build, test and deploy on every push.
Version control	Git + GitHub	Code management and team collaboration.

2. Summary

The whole stack is built on the **Microsoft / .NET** ecosystem, from the UI down to deployment. This reduces integration problems, simplifies deployment to Azure, and covers every project requirement: booking and calendar, real-time and external notifications, medical records and prescriptions, and reports and analytics.

14. Deployment Diagram

Deployment view



MediCare deployment on Azure · 6 nodes, 2 external services, 1 pipeline

Arrows show who calls whom. Browsers load pages from the App Service over HTTPS and keep live-notification connections open to Azure SignalR Service. The pipeline deploys to the staging slot first, and the tested build is swapped into production.

Nodes and artifacts

Node	What runs on it	Notes
User device (browser)	Razor views, Bootstrap 5, FullCalendar.js, Chart.js, SignalR JavaScript client	Patients, doctors and admins all use a browser. No install.

MediCare runs on Microsoft Azure: ASP.NET Core MVC app on Azure App Service, data in Azure SQL Database, real-time notifications through Azure SignalR Service, and a GitHub Actions / Azure DevOps

pipeline. SMS and email go through Twilio and SendGrid. The three N-Tier projects compile into **one deployable web app**, so they share a single App Service node and are separated by code layers, not servers.

Nodes and artifacts

Node	What runs on it	Notes
User device (browser)	Razor views, Bootstrap 5, FullCalendar.js, Chart.js, SignalR JS client	No install
Azure App Service (Web App)	MediCare.Web, MediCare.Services, MediCare.Data (repositories, Unit of Work, EF Core, Identity)	One deployable; reminder scheduling as hosted background service
App Service staging slot	Same artifact, pre-production build	Tests run here before slot swap
Azure SQL Database	Schema from EF Core migrations	Only App Service connects
Azure SignalR Service	Managed WebSocket connections	App Service hosts the hub
Azure Blob Storage *(recommended addition)*	Test results, X-rays, medical documents	Not in the brief; only the file path is stored in SQL
Twilio (external)	SMS reminders	HTTPS from Services layer
SendGrid (external)	Email notifications	HTTPS from Services layer
GitHub Actions / Azure DevOps	Build, test, package, deploy	Staging first, then production

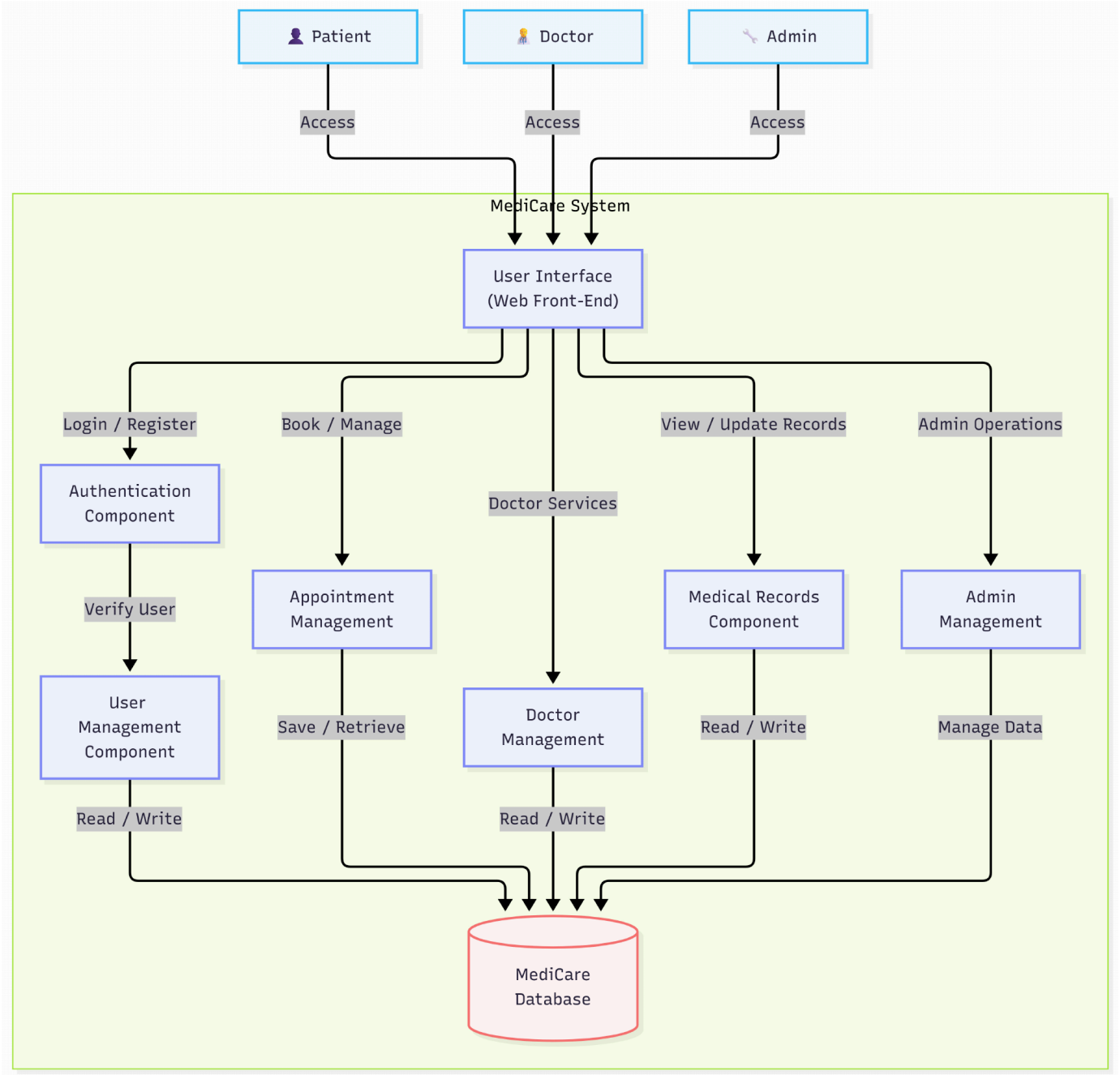
Communication paths

From	To	Protocol	Purpose
Browser	App Service	HTTPS (TLS)	Pages, form posts, AJAX for FullCalendar/Chart.js
Browser	Azure SignalR Service	WebSocket over TLS	Live notifications
App Service	Azure SignalR Service	HTTPS / WebSocket	Hub publishes messages
App Service	Azure SQL Database	TDS over TLS (EF Core)	Queries and transactions
App Service	Azure Blob Storage	HTTPS	Upload/download medical documents
App Service	Twilio	HTTPS REST	SMS reminders 24 h before
App Service	SendGrid	HTTPS REST	Email confirmations and reminders
CI/CD pipeline	App Service staging slot	HTTPS (publish profile / service connection)	Deploy, then swap

Environments and CI/CD

1. Push/PR triggers the pipeline: restore, build, run unit and integration tests.
2. Publish and deploy to the **staging slot**, applying EF Core migrations to the staging database.
3. Smoke tests and key end-to-end tests run on staging.
4. After approval, swap the slot into production.
5. Twilio/SendGrid keys and the SQL connection string live in App Service configuration or Key Vault — never in source control.

15. Component Diagram



Component	Connected to	Flow label
Patient, Doctor, Admin (actors)	User Interface (Web Front-End)	Access
User Interface (Web Front-End)	Authentication Component	Login / Register
	Appointment Management	Book / Manage
	Doctor Management	Doctor Services
	Medical Records Component	View / Update Records
	Admin Management	Admin Operations
Authentication Component	User Management Component	Verify User
User Management Component	MediCare Database	Read / Write
Appointment Management	MediCare Database	Save / Retrieve
Doctor Management	MediCare Database	Read / Write
Medical Records Component	MediCare Database	Read / Write

Component	Connected to	Flow label
Admin Management	MediCare Database	Manage Data